PYTHON ACTIVITY BASED LEARNING

PROBLEM—10

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

Objective:

To develop a University Scholarship Allocation System that evaluates student applications based on aggregate marks, subject-wise minimum scores, attendance requirements, and family income criteria. The system ranks eligible students according to predefined rules, resolves ties fairly, and allocates scholarships within the available quota.

Algorithm:

1. Start.

2. Input the number of scholarship applicants.

3. For each applicant, input applicant ID, name, aggregate marks, marks in three subjects, attendance percentage, and annual family income.

4.Input the minimum aggregate marks, minimum subject marks, minimum attendance, and maximum income criteria.

5.Initialize two lists: eligible and rejected.

6.Check each applicant against all eligibility conditions:

- Aggregate marks must meet the minimum requirement.
- Every subject mark must meet the minimum subject score.
- Attendance must meet the minimum requirement.
- Annual family income must not exceed the maximum limit.

7.Add applicants who satisfy all conditions to eligible; otherwise, add them to rejected.

8.Sort eligible applicants according to these tie-breaking rules:

- Higher aggregate marks first.
- Higher attendance first.
- Lower family income first.
- Lower applicant ID first.

9.Display the ranked list of eligible applicants.

10.Display the rejected applicants.

11. Display the total, eligible, and rejected applicant counts.

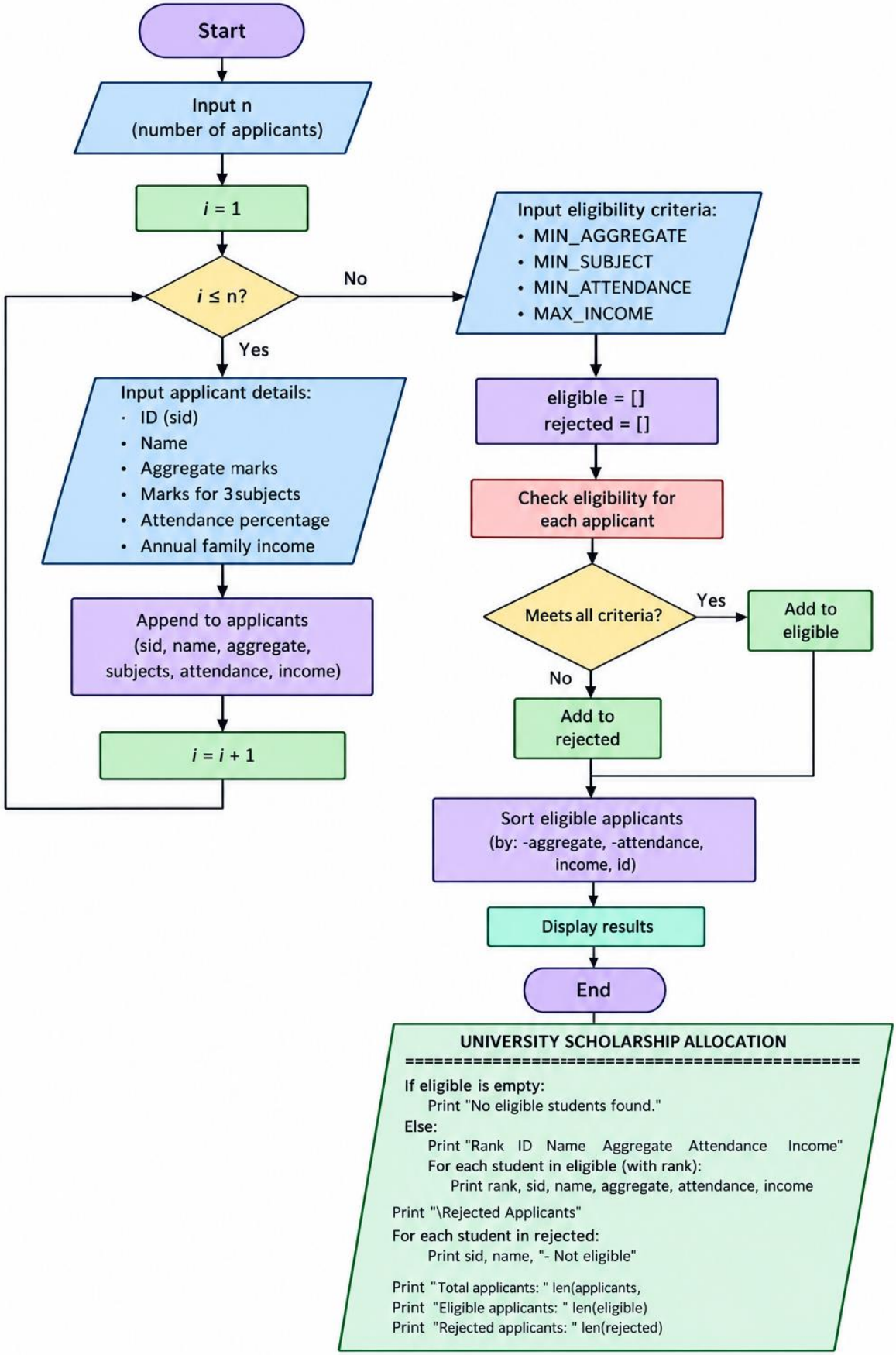
12. Stop.

Pseudocode:

```
BEGIN
INPUT number_of_applicants
INITIALIZE applicants = empty list
FOR each applicant
    INPUT applicant_ID
    INPUT name
    INPUT aggregate_marks
    INPUT marks for three subjects
    INPUT attendance
    INPUT annual_family_income
    STORE applicant details
END FOR
INPUT minimum_aggregate
INPUT minimum_subject_marks
INPUT minimum_attendance
INPUT maximum_income
INITIALIZE eligible = empty list
INITIALIZE rejected = empty list
// Stage 1: Eligibility checking
FOR each applicant in applicants
    IF aggregate_marks >= minimum_aggregate AND
       ALL subject marks >= minimum_subject_marks AND
       attendance >= minimum_attendance AND
```

```
        annual_family_income <= maximum_income THEN
         ADD applicant to eligible
      ELSE
         ADD applicant to rejected
      END IF
   END FOR
   // Stage 2: Ranking
   SORT eligible BY
      aggregate marks in descending order,
      attendance in descending order,
      income in ascending order,
      applicant ID in ascending order
   // Stage 3: Display results
   IF eligible list is empty THEN
      DISPLAY "No eligible students found"
   ELSE
      DISPLAY ranked eligible applicants
   END IF
   DISPLAY rejected applicants
   DISPLAY total applicants
   DISPLAY eligible applicants count
   DISPLAY rejected applicants count
   STOP
```

Flowchart:


Start
Input n
(number of applicants)
i = 1
i ≤ n?
No
Yes
Input applicant details:
• ID (sid)
• Name
• Aggregate marks
• Marks for 3 subjects
• Attendance percentage
• Annual family income
Append to applicants
(sid, name, aggregate,
subjects, attendance, income)
i = i + 1
Input eligibility criteria:
• MIN_AGGREGATE
• MIN_SUBJECT
• MIN_ATTENDANCE
• MAX_INCOME
eligible = []
rejected = []
Check eligibility for
each applicant
Meets all criteria?
Yes
Add to
eligible
No
Add to
rejected
Sort eligible applicants
(by: -aggregate, -attendance,
income, id)
Display results
End
UNIVERSITY SCHOLARSHIP ALLOCATION
==========
If eligible is empty:
Print "No eligible students found."
Else:
Print "Rank ID Name Aggregate Attendance Income"
For each student in eligible (with rank):
Print rank, sid, name, aggregate, attendance, income
Print "\Rejected Applicants"
For each student in rejected:
Print sid, name, "- Not eligible"
Print "Total applicants: " len(applicants,
Print "Eligible applicants: " len(eligible)
Print "Rejected applicants: " len(rejected)

Python Program:

```
# University Scholarship Allocation
n = int(input("Enter number of applicants: "))
applicants = []
for i in range(n):
    print("\nApplicant", i + 1)
    sid = int(input("Enter applicant ID: "))
    name = input("Enter applicant name: ")
    aggregate = float(input("Enter aggregate marks: "))
    subjects = []
    for j in range(3):
        mark = float(input(
            "Enter marks for subject " + str(j + 1) + ": "
        ))
        subjects.append(mark)
    attendance = float(input("Enter attendance percentage: "))
    income = float(input("Enter annual family income: "))
    applicants.append(
        (sid, name, aggregate, tuple(subjects), attendance, income)
    )
# Eligibility criteria
MIN_AGGREGATE = float(input("\nMinimum aggregate marks: "))
MIN_SUBJECT = float(input("Minimum subject marks: "))
MIN_ATTENDANCE = float(input("Minimum attendance percentage: "))
MAX_INCOME = float(input("Maximum annual family income: "))
eligible = []
rejected = []
# Stage 1: Check eligibility
for student in applicants:
```

```
    sid, name, aggregate, subjects, attendance, income = student
    if (aggregate >= MIN_AGGREGATE
        and all(mark >= MIN_SUBJECT for mark in subjects)
        and attendance >= MIN_ATTENDANCE
        and income <= MAX_INCOME):
        eligible.append(student)
    else:
        rejected.append(student)
# Stage 2: Rank eligible applicants
# 1. Higher aggregate
# 2. Higher attendance
# 3. Lower family income
# 4. Lower applicant ID
eligible.sort(key=lambda s: (-s[2], -s[4], s[5], s[0]))
# Stage 3: Display results
print("\nUNIVERSITY SCHOLARSHIP ALLOCATION")
print("=" * 65)
if not eligible:
    print("No eligible students found.")
else:
    print("Rank ID Name Aggregate Attendance Income")
    for rank, student in enumerate(eligible, start=1):
        sid, name, aggregate, subjects, attendance, income = student
        print(rank, sid, name, aggregate, attendance, income)
print("\nRejected Applicants")
for student in rejected:
    print(student[0], student[1], "- Not eligible")
 print("\nTotal applicants:", len(applicants))
print("Eligible applicants:", len(eligible))
```

print("Rejected applicants:", len(rejected))

Output Screen Shots:

